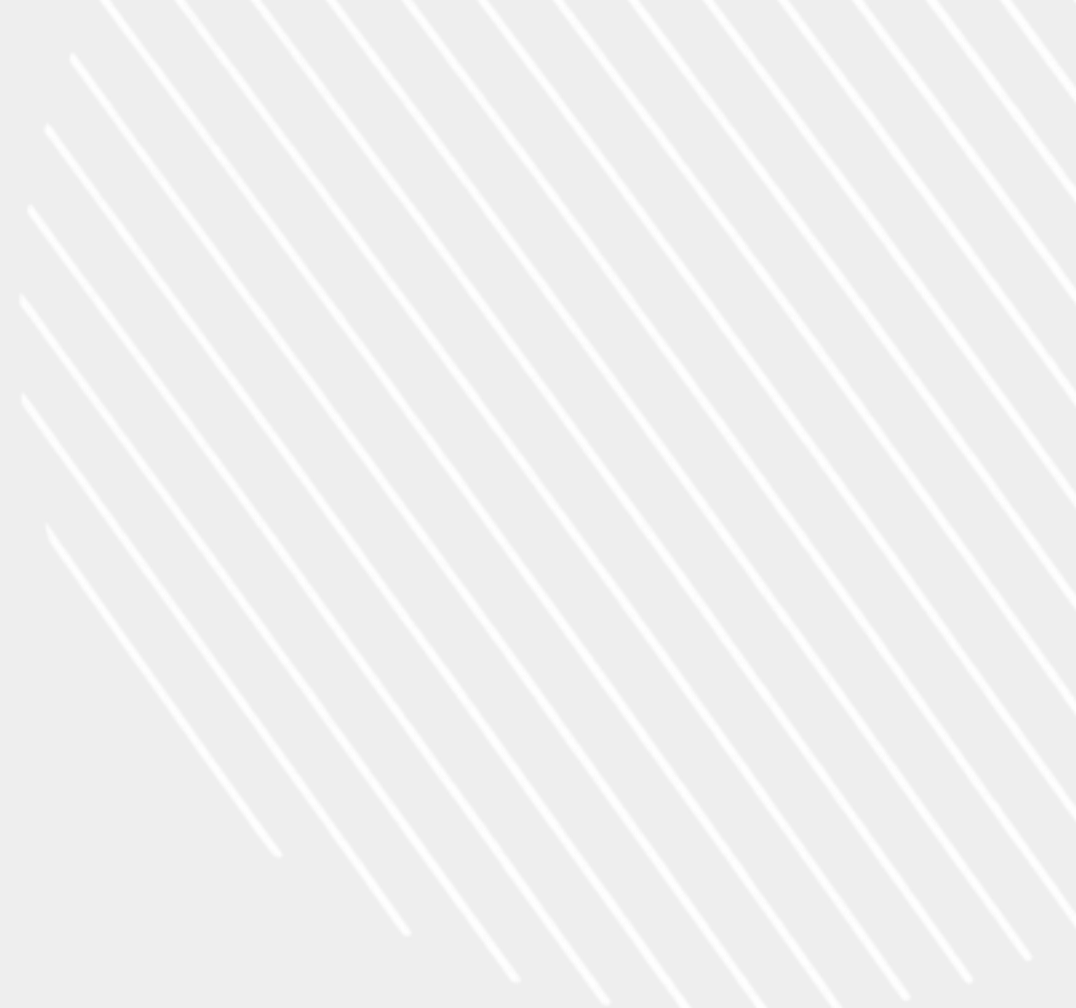
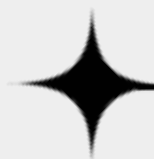




PORTFOLIO



게임 클라이언트 프로그래머 장명운



CONTACT

+82 010 6477 4881

wkdauddns9@gmail.com

CONTENTS

01

PROFILE

02

VISION

03

PROJECT | 졸업작품 <이방인(Outlander)>

04

PROJECT | 윈도우 프로그래밍 텀프로젝트 <Dungreed 모작>

05

PROJECT | 스크립트 언어 텀프로젝트 <Papery>



01 PROFILE



장명운

Jang Myeong Woon

About

1999.12.09

wkdauddns9@gmail.com

010-6477-4881

학력사항

2018.02. 양서고등학교 졸업

2020.03. 한국공학대학교 게임공학과 입학

2027.02. 한국공학대학교 게임공학과 졸업(예정)

활동

2024.03.-2024.06.

학과 학술 소모임 WARP, C++ 멘토 활동

2024.09.-2025.12.

학과 '컴퓨터 그래픽스' 과목 조교 활동

2025.03.-2026.06.

학과 '윈도우 프로그래밍' 과목 조교 활동

2026.04.-2026.05.

한국게임학회 2026 춘계학술대회 논문 발표

참여 논문 'Hi-Z 오클루전 컬링 결과를 활용한 가시성 기반 애니메이션 갱신 기법'

프로젝트

2021.05.-2021.06.

Win32 API를 활용한 작품 <던그리드> 모작

2024.05.-2024.06.

논문 검색 및 AI 요약 프로그램 <Papery> 개발

2024.09.-2026.06.

졸업작품 <이방인(Outlander)> 개발

수상 이력

- 2021학년도 한국공학대학교 게임공학과 과제전 '윈도우 프로그래밍' 부문 1위
- 2026학년도 한국공학대학교 게임공학과 졸업작품 총장상 수상

02 VISION

"저의 비전은 이렇습니다."

"좋은 코드"를 시간에 따라 변화하는 팀원의 구성과 기술적 역량, 프로젝트의 특성 등에 적응하여, "팀원들이 최소한의 작업 부담을 느끼며 시간 효율적으로 확장할 수 있는 코드"라 생각하는 동시에 지향합니다.

최종 목표를 위한 단위 작업을 설계하고, 팀원에게 할당하는 역할을 많이 수행해왔습니다. 파일 포맷 설계, 툴 제작, 코딩 패턴 설계 등 개발 중간 과정의 효율을 올리기 위한 일련의 작업들을 무엇보다 중요하게 생각합니다.

주요 연구 분야

- Direct3D 12 렌더링 엔진 설계
- 게임 클라이언트 멀티스레드 최적화
- 제약 조건 기반 물리 처리
- 스켈레탈 애니메이션 블렌딩 및 최적화
- GPU 주도 렌더링

03 PROJECT | 이방인(Outlander)



이방인(Outlander)

2026 한국공학대학교 게임공학과 졸업작품

장르 : 4인 co-op 판타지 무쌍 액션 게임

기획 의도 : '진 · 삼국무쌍' 시리즈처럼 많은 적들이 등장해
일당백 액션을 체험할 수 있는 판타지 배경 멀티 플레이 게임

사용 언어 : C++20, C#, HLSL, Python, Lua

개발 환경 : Visual Studio 2022, Direct3D12, Unity, PIX, Git

핵심 연구 과제 설정

- Ragdoll Physics | 화면 상 다수의 적들이 전부 같은 모션으로 죽으면 단조롭고 부자연스러울 것이라 예상해 설정
- Seamless Chunk Loading | 액션이 중시되는 만큼 로딩으로 인해 흐름이 끊기지 않도록 하기 위해 설정
- Strategic AI Enemy | 단순히 숫자만 많을 뿐인 적보다, 진형을 갖추고 전술을 활용하는 적이
몰입감을 높여줄 것이라 판단해 설정

개발 인원 : 3인

담당 역할 : 메인 클라이언트 프로그래머 및 서버 서버 프로그래머

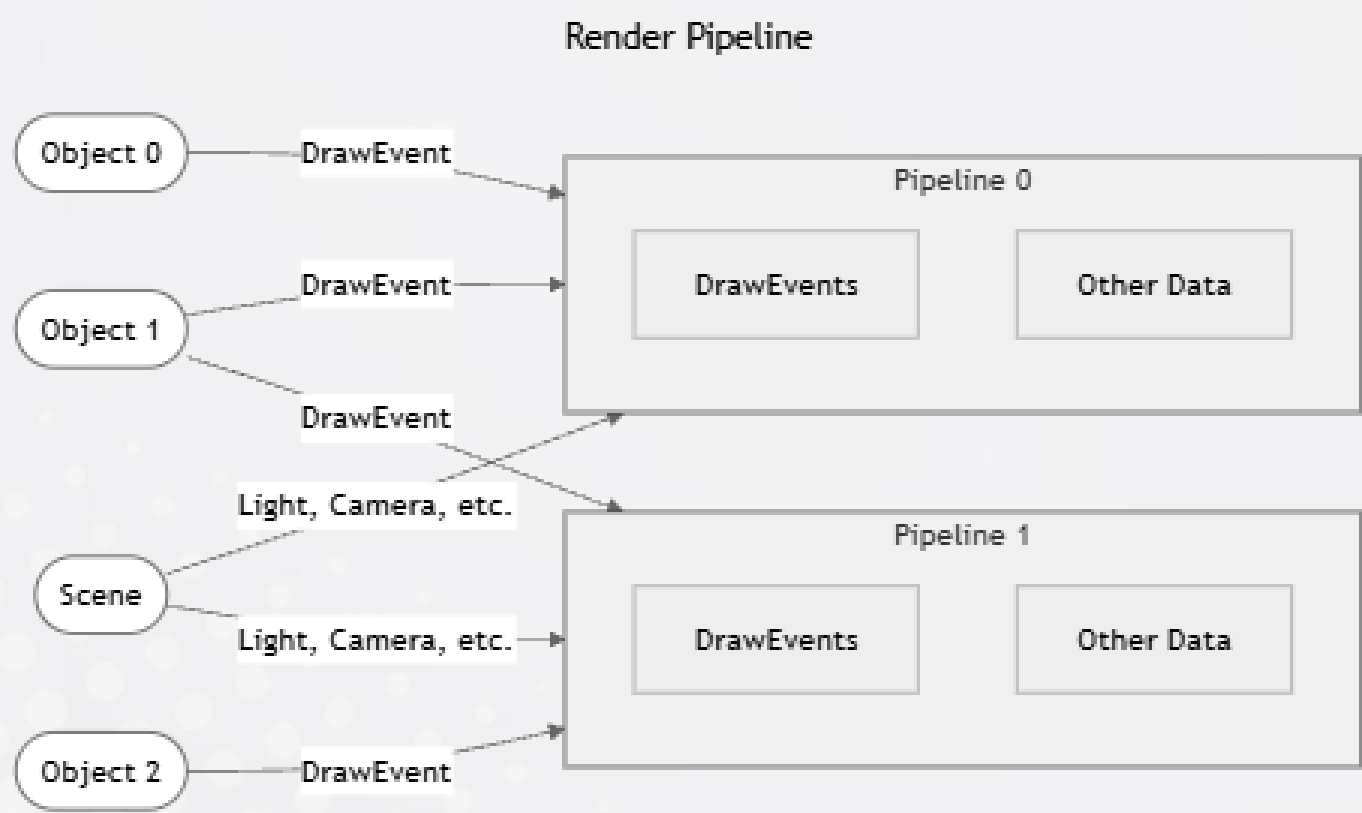
프로젝트 단위 작업 정의 및 리딩, 모든 개별 구현 merge에 대한 최종 검토, 렌더링 파이프라인 패턴화,
주요 3D 렌더링 파이프라인 구현, 애니메이션 및 물리 전반 구현, 레벨 제작, 캐릭터 리소스 변환 및 추출,
스킬 시스템 구현

03 PROJECT | 이방인(Outlander) – 주요 기여

Rendering Pipeline Design

그래픽스를 총괄하는 클래스인 GFX 내에
파이프라인별 D3D 리소스 멤버, 렌더링 데이터 컨테이너 보관

오브젝트 클래스들은 Object::render(GFX&)을 override하여
특정한 파이프라인에 DrawEvent 구조체를 제출



```
for (std::size_t i = 0u; i < mesh.subMeshes.size(); ++i) {
    if (useDeferred) {
        if (isSkinned) {
            auto de = PBRDeferredSkinnedPipeline::DrawEvent{
                .world          = meshXform * offsetXform * renderState_.world,
                .boneXforms     = renderState_.animBlender->finalXformData(),
                .mesh            = &mesh,
                .subMesh         = &mesh.subMeshes[i],
                .material        = &mesh.materialSets[materialSetIdx_].materials
                .renderObjectId  = renderObjectId_,
                .bakedClipId     = bakedReady
                    ? renderState_.animBlender->finalBakedClipId() : -1,
                .bakedClipFrame  = bakedReady
                    ? renderState_.animBlender->finalBakedClipFrame() : -1,
                .viewFrustumCulled = culled,
                .shadowCulled     = shadowCulled,
                .worldCullMin     = cullMin,
                .worldCullMax     = cullMax,
                .hasWorldCullBounds = hasCullBounds,
            };
            // Energy-orb absorption ripples (local player body wave). Empty for
            // every other skinned object -> rippleCount 0 -> no shader cost.
            const int nr = std::min<int>(static_cast<int>(bodyRipples_.size()),
                PBRDeferredSkinnedPipeline::DrawEvent::kMaxRipples);
            for (int r = 0; r < nr; ++r) {
                const auto& br = bodyRipples_[r];
                // Re-anchor to the live body position so the ring tracks the play
                de.ripplePosAge[r] = mu::Vec4(renderState_.pos + br.offset
                    de.rippleColorIntensity[r] = mu::Vec4(br.colorHDR, br.intensity);
            }
            de.rippleCount = static_cast<u32t>(nr);
            gfx.addDrawEvent(std::move(de));
        }
        else {
            gfx.addDrawEvent(PBRDeferredPipeline::DrawEvent{
                .world          = meshXform * offsetXform * renderState_.world,
                .mesh            = &mesh,
                .subMesh         = &mesh.subMeshes[i],
                .material        = &mesh.materialSets[materialSetIdx_].materials
                .viewFrustumCulled = culled,
                .shadowCulled     = shadowCulled,
            });
        }
    }
    else {
        if (isSkinned) {
            // 드로우콜 요청을 제출한다. render() 호출 시 그려진다.
            void addDrawEvent(const SamplePipeline::DrawEvent& drawEvent);
            // 카메라 데이터를 입력한다.
            void addCameraData(const SamplePipeline::CameraData& cameraData);
            // 드로우콜 요청을 제출한다. render() 호출 시 그려진다.
            void addDrawEvent(const PBRPipeline::DrawEvent& drawEvent);
            // 카메라 데이터를 입력한다.
            void addCameraData(const PBRPipeline::CameraData& cameraData);
            // 조명 데이터를 입력한다.
            void addLightData(const PBRPipeline::LightData& lightData);
            // 프레임 데이터를 입력한다.
            void addFrameData(const PBRPipeline::FrameData& frameData);
            // 드로우콜 요청을 제출한다. render() 호출 시 그려진다.
            void addDrawEvent(const PBRSkinnedPipeline::DrawEvent& drawEvent);
            // 카메라 데이터를 입력한다.
            void addCameraData(const PBRSkinnedPipeline::CameraData& cameraData);
            // 조명 데이터를 입력한다.
            void addLightData(const PBRSkinnedPipeline::LightData& lightData);
            // 프레임 데이터를 입력한다.
            void addFrameData(const PBRSkinnedPipeline::FrameData& frameData);
            // 드로우콜 요청을 제출한다. render() 호출 시 그려진다.
            void addDrawEvent(const PBRDeferredPipeline::DrawEvent& drawEvent);
            // 카메라 데이터를 입력한다.
            void addCameraData(const PBRDeferredPipeline::CameraData& cameraData);
            // 조명 데이터를 입력한다.
            void addLightData(const PBRDeferredPipeline::LightData& lightData);
            // 프레임 데이터를 입력한다.
            void addFrameData(const PBRDeferredPipeline::FrameData& frameData);
            // 드로우콜 요청을 제출한다. render() 호출 시 그려진다.
            void addDrawEvent(const PBRDeferredSkinnedPipeline::DrawEvent& drawEvent);
            // 카메라 데이터를 입력한다.
            void addCameraData(const PBRDeferredSkinnedPipeline::CameraData& cameraData);
            // 조명 데이터를 입력한다.
            void addLightData(const PBRDeferredSkinnedPipeline::LightData& lightData);
            // 프레임 데이터를 입력한다.
            void addFrameData(const PBRDeferredSkinnedPipeline::FrameData& frameData);
        }
    }
}
```

패턴화된 파이프라인별 렌더링 데이터 제출 방식

Object::render(GFX&) 내부 구현

03 PROJECT | 이방인(Outlander) – 주요 기여

Rendering Pipeline Design

각 렌더링 파이프라인은 렌더 패스를 public 함수로
제공하여 명령 리스트 풀에서 명령 리스트를 할당받아
렌더링 명령을 기록하고 실행하는 일련의 과정 정의

CPU 작업 시간이 긴 렌더 패스들에 대해
DrawEvent들을 그룹화, ThreadPool에 Job 제출

트리플 버퍼링을 통해 CPU-GPU 동기화 지연 최소화
정점 버퍼, 인덱스 버퍼와 같이 정적인 데이터는 1개,
오브젝트 변환 정보와 같이 동적인 데이터는 3개의
D3D12 리소스를 마련해 백버퍼별로 사용

프레임 버퍼 동기화를 위한 Fence에
사용된 명령 리스트와 별도로 해제할 리소스를 연관시켜
리소스 해제 과정이 gpu 작업의 종료 이후에
이루어질 것을 보장

```
void Dispatcher::gBufferDrawMT() {
    if (gBufferEvents_.empty()) return;

    static auto instancingGroups = std::vector<decltype(gBufferEvents_)::const_iterator>();
    auto it = gBufferEvents_.cbegin();
    while (it != gBufferEvents_.cend()) {
        instancingGroups.push_back(it);
        it = std::upper_bound(it, gBufferEvents_.cend(), *it);
    }
    instancingGroups.push_back(gBufferEvents_.cend());

    const std::size_t jobCnt = ((instancingGroups.size() - 1u) + (jobSizeDraw_ - 1u)) / jobSizeDraw_;
    auto latch = std::latch(jobCnt);

    std::list<CommandContext> cmdCtxs{};
    const auto allocCnt = cmdListPool_>alloc(jobCnt, CommandListUsage::RenderingSlave, cmdCtxs);
    DISPLAY_ERROR_STR(allocCnt == jobCnt,
        "[GFX Error] PBRDeferredSkinnedPipeline::gBufferDrawMT: not enough command lists.", false);
    if (allocCnt != jobCnt) {
        cmdListPool_>free(CommandListUsage::RenderingSlave, std::move(cmdCtxs));
        instancingGroups.clear();
        return;
    }

    std::size_t accDC = 0u;
    auto currCtx = cmdCtxs.begin();

    while (accDC + (jobSizeDraw_ - 1) < instancingGroups.size() - 1u) {
        DISPLAY_ERROR_DX_HR(currCtx->cmdAlloc->Reset(), false);
        DISPLAY_ERROR_DX_HR(currCtx->cmdList->Reset(currCtx->cmdAlloc.Get(), nullptr), false);
        addJobGBufferDraw(currCtx->cmdList.Get(), instancingGroups.data() + accDC,
            instancingGroups.data() + accDC + jobSizeDraw_, accDC, latch);
        accDC += jobSizeDraw_;
        ++currCtx;
    }

    if (accDC != instancingGroups.size() - 1u) {
        const auto last = instancingGroups.size() - 1u - accDC;
        DISPLAY_ERROR_DX_HR(currCtx->cmdAlloc->Reset(), false);
        DISPLAY_ERROR_DX_HR(currCtx->cmdList->Reset(currCtx->cmdAlloc.Get(), nullptr), false);
        addJobGBufferDraw(currCtx->cmdList.Get(), instancingGroups.data() + accDC,
            instancingGroups.data() + accDC + last, accDC, latch);
    }

    latch.wait();
    instancingGroups.clear();

    auto staged = std::vector<ID3D12CommandList*>();
    staged.reserve(cmdCtxs.size());
    for (auto& ctx : cmdCtxs) staged.push_back(ctx.cmdList.Get());
    DISPLAY_ERROR_DX_VOID(submitter_>submit(static_cast<UINT>(staged.size()), staged.data(), false);
    pFence_>associatedCmdCtxs_[etoi(CommandListUsage::RenderingSlave)]
        .splice(pFence_>associatedCmdCtxs_[etoi(CommandListUsage::RenderingSlave)].end(), std::move(cmdCtxs));
}

while (pGroup != pItLast) {
    auto gFirst = *pGroup;
    auto gLast = *(pGroup + 1);
    const auto& de = *gFirst;

    threadCmdList->SetGraphicsRoot32BitConstant( rootParamIdxFirstInstOffset_,
        static_cast<u32t>(gFirst - gBufferEvents_.begin()), 0u
    );

    pResources_>gBufferPass.perDrawcallData.cbuffers[idxDC].bind(
        threadCmdList, rootParamIdxPDD_, roomId_);

    auto pdd = PBRDeferredSkinnedGBufferShader::PerDrawcallData{
        .material = PBRDeferredSkinnedGBufferShader::Material{
            .idxAlbedo = de.material->mapAlbedo.idxSrv,
            .idxMetallicSmoothness = de.material->mapMetallicSmoothness.idxSrv,
            .idxNormal = de.material->mapNormal.idxSrv,
            .idxEmmissive = de.material->mapEmmissive.idxSrv,
            .idxAmbientOcclusion = de.material->mapAmbientOcclusion.idxSrv,
            .cAlbedo = de.material->constantAlbedo,
            .cRoughness = de.material->constantRoughness,
            .cMetallic = de.material->constantMetallic,
            .cAOSTrength = de.material->constantAOSTrength,
            .cAlphaCutoff = de.material->constantAlphaCutoff,
            .cEmmissive = de.material->constantEmmissive
        }
    };
    pResources_>gBufferPass.perDrawcallData.cbuffers[idxDC].stage(roomId_, &pdd, 1u);

    layoutMeshIfNeeded(*de.mesh);
    auto& vbViews = de.mesh->vbViewsByPipeline.at("PBRDeferredSkinnedPipeline_GBuffer");
    DISPLAY_ERROR_DX_VOID(threadCmdList->IASetVertexBuffers(
        0u, static_cast<UINT>(vbViews.size()), vbViews.data(), false);
    DISPLAY_ERROR_DX_VOID(threadCmdList->IASetIndexBuffer(&de.subMesh->ibView), false);

    const auto stride = de.subMesh->ibView.Format == DXGI_FORMAT_R16_UINT ? sizeof(u16t) : sizeof(u32t);
    DISPLAY_ERROR_DX_VOID( threadCmdList->DrawIndexedInstanced(
        static_cast<UINT>(de.subMesh->ibView.SizeInBytes / stride),
        static_cast<UINT>(gLast - gFirst), 0u, 0, 0u
    ), false );

    ++idxDC;
    ++pGroup;
}
DISPLAY_ERROR_DX_HR(threadCmdList->Close(), false);
latch.count_down();
```

렌더링 파이프라인 내부 구현 – Master Thread

렌더링 파이프라인 내부 구현 – Slave Thread

03 PROJECT | 이방인(Outlander) – 주요 기여

Physics System

물리량 적분 및 충돌 판정, 제약 조건 처리를 총괄하는
PhysicsWorld 클래스 작성

고정 주기 업데이트를 통해 카오스 방지

PhsyicsWorld::step pseudo code

For each substep:

integrate(sub Δt)

broad phase contact search // SAP-based

narrow phase contact search // BVH to BVH

generate contact constraints

for each velocitySolverIteration:

 solveVelocity()

for each positionSolverIteration:

 solvePosition()

resolveStaticPenetration()

```
void PhysicsWorld::integrate(Seconds dt)
{
    for (auto& e : entries_) {
        RigidBody& b = *e.body;

        switch (b.motionType()) {
            case MotionType::Kinematic: { ... }
            case MotionType::Dynamic:
            {
                if (b.invMass() <= 0.f) break; // guard: treat as Static if mass unset

                const float dtf = dt.count();

                // Velocity damping (applied first so it does not damp the new impulse).
                // x/z: ground friction (linearDamping). y: air resistance (kAirDamping).
                {
                    const auto vel = b.linearVel();
                    const float horzDamp = std::max(0.f, 1.f - b.linearDamping() * dtf);
                    const float vertDamp = std::max(0.f, 1.f - kAirDamping * dtf);
                    b.setLinearVel(mu::Vec3(vel.x() * horzDamp, vel.y() * vertDamp, vel.z() * horzDamp));
                }
                b.setOmega(b.omega() * std::max(0.f, 1.f - b.angularDamping() * dtf));

                // Per-body gravity gate: grounded character controllers set
                // gravityScale to 0 so gravity stops fighting the contact solver
                // (no residual vertical micro-jitter). Restored to 1 when airborne.
                const auto linAcc = gravity_ * b.gravityScale() + b.forceAccum() * b.invMass();
                b.setLinearVel(b.linearVel() + linAcc * dtf);

                // Integrate angular velocity: I_world^-1 * torque.
                // In DirectXMath row-vector form: torque * invInertiaWorld.
                b.updateInertiaWorld();
                const auto angAcc = b.torqueAccum() * b.invInertiaWorld();
                b.setOmega(b.omega() + angAcc * dtf);

                // Upright correction: apply a restoring torque that pulls the body's
                // local Y-axis back toward world Y-up (gravity's reference direction).
                // cross(bodyUp, worldUp) points along the axis that rotates bodyUp onto
                // worldUp; its magnitude equals sin(tilt angle) so the correction is
                // proportional to how far the body has tilted.
                if (b.uprightStiffness() > 0.f) {
                    const mu::Vec3 bodyUp = b.orient().rotate(mu::Vec3(0.f, 1.f, 0.f));
                    const mu::Vec3 torqDir = mu::cross(bodyUp, mu::Vec3(0.f, 1.f, 0.f));
                    b.setOmega(b.omega() + (torqDir * (b.uprightStiffness() * dtf)) * b.invInertiaWorld());
                }

                // Clamp angular speed to prevent runaway spin (e.g. from unsolved joint chains).
                {
                    static constexpr float kMaxAngularSpeed = 50.f;
                    const float omegaLen2 = b.omega().len2();
                    if (omegaLen2 > kMaxAngularSpeed * kMaxAngularSpeed)
                        b.setOmega(b.omega() * (kMaxAngularSpeed / std::sqrt(omegaLen2)));
                }

                // [SAFETY 1] Linear speed clamp + NaN guard (see client/docs/ragdollSafety.md).
                // Matches the angular clamp above so runaway translation can't jump positions
                // and cascade into NaN; the guards keep a single non-finite value from
                // corrupting the body (and, once rendered, risking a device-removed crash).
                {
                    static constexpr float kMaxLinearSpeed = 40.f;
                    const float velLen2 = b.linearVel().len2();
                    if (velLen2 > kMaxLinearSpeed * kMaxLinearSpeed)
                        b.setLinearVel(b.linearVel() * (kMaxLinearSpeed / std::sqrt(velLen2)));
                }
            }
        }
    }
}
```

Integrate(sub Δt)

```
void PhysicsWorld::solveConstraints(Seconds dt)
{
    auto allConstraints = [&](auto fn) {
        for (auto& c : contactConstraints_) fn(*c);
        for (auto& c : jointConstraints_)   fn(*c);
        for (auto c : jointRefs_)           fn(*c);
    };

    allConstraints([&](Constraint& c) { c.prepare(dt); });

    // Warm-start: apply previous step's accumulated impulses as initial guess.
    // Called after prepare() so rA/rB/tangent are fresh.
    for (auto& cc : contactConstraints_) {
        auto it = warmCache_.find(normKey(cc->bodyA, cc->bodyB));
        if (it != warmCache_.end()) {
            const auto& w = it->second;
            for (int i = 0; i < cc->count; ++i) {
                cc->contacts[i].accNormal = w.accNormal;
                cc->contacts[i].accTangent[0] = w.accTangent[0];
                cc->contacts[i].accTangent[1] = w.accTangent[1];
            }
            cc->applyWarmStart();
        }
    }

    for (int iter = 0; iter < solverIterations_; ++iter)
        allConstraints([&](Constraint& c) { c.solveVelocity(); });

    for (int iter = 0; iter < jointSolverExtraIterations_; ++iter) {
        for (auto& c : jointConstraints_) c->solveVelocity();
        for (auto c : jointRefs_)         c->solveVelocity();
    }

    for (int iter = 0; iter < positionSolveIterations_; ++iter)
        allConstraints([&](Constraint& c) { c.solvePosition(); });

    // Save accumulated impulses for next step's warm-start.
    warmCache_.clear();
    for (auto& cc : contactConstraints_) {
        if (cc->count == 0) continue;
        WarmEntry w;
        w.accNormal = cc->contacts[0].accNormal;
        w.accTangent[0] = cc->contacts[0].accTangent[0];
        w.accTangent[1] = cc->contacts[0].accTangent[1];
        warmCache_[normKey(cc->bodyA, cc->bodyB)] = w;
    }
}
```

solver

03 PROJECT | 이방인(Outlander) – 주요 기여

Bounding Volume Hierarchy

LOD별 Bounding Volume 그룹 생성

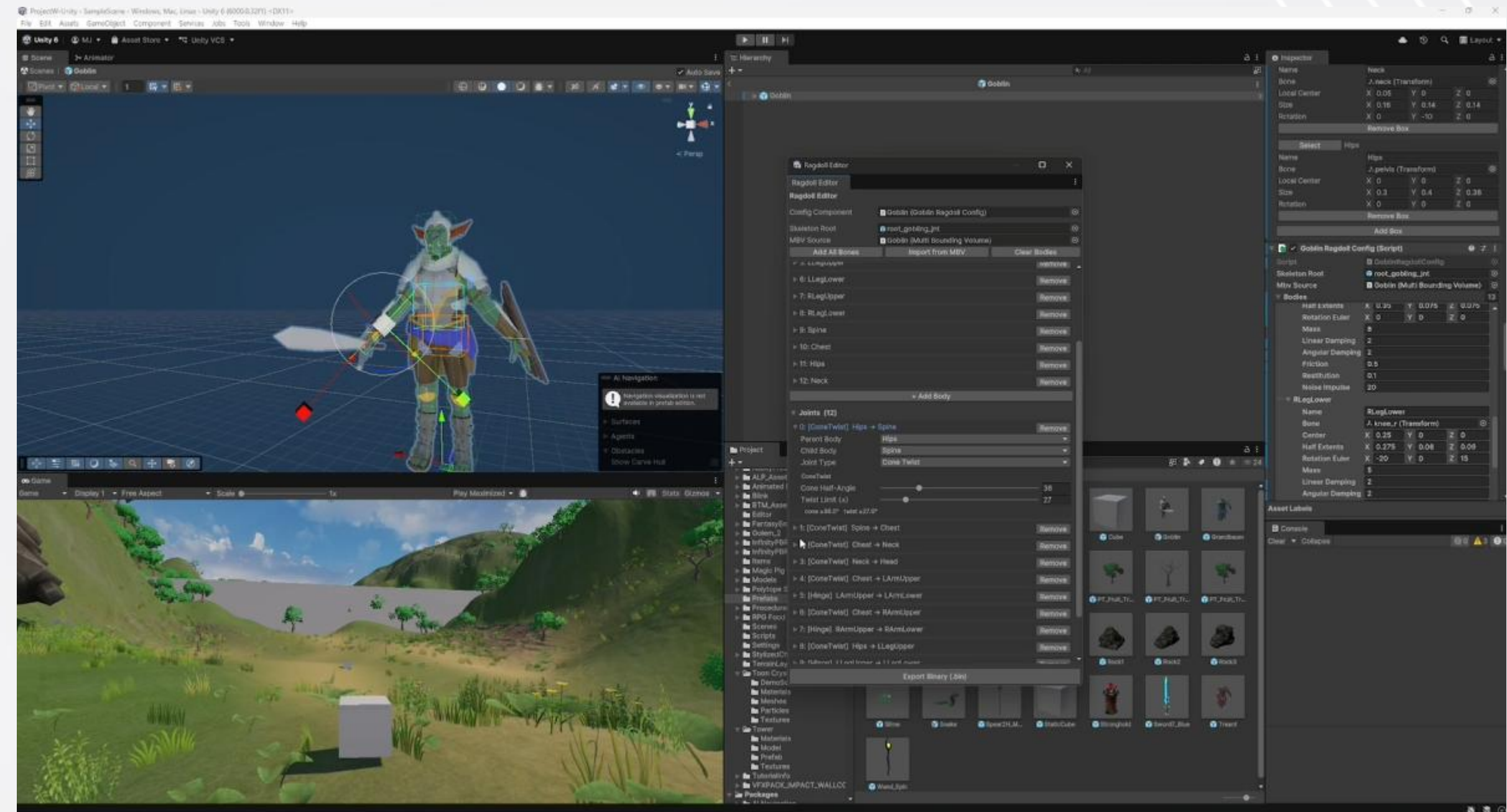
Ragdoll Body와 LOD 1의 Bounding Volume 일대일 대응

BVH vs BVH 충돌 판정 시 스택에 각 루트 노드의 pair 삽입,
스택에서 추출한 pair가 서로 겹칠 경우 둘 중 부피가 큰 노드를
그 자식 노드로 교체한 pair들을 스택에 삽입

리프 노드들에 도달하기까지 두 Bounding Volume이 겹쳤다면
충돌 확정

충돌 판정 최적화 효과

⇒ Ragdoll Simulation, 부위별 피격 데미지
구현에 활용



Bounding Volume Hierarchy, Ragdoll Body 에디터 GUI

03 PROJECT | 이방인(Outlander) – 주요 기여

Ragdoll Physics

적 몬스터 사망 시 현재 애니메이션 포즈에서 래그돌화

주요 Bone → Physic Body와 일대일 대응

나머지 Bone → 근접한 주요 Bone에 대해

애니메이션 포즈 상 상대 변환을 적용

유니티에서 GUI를 통해 Physic Body 제작 및 추출

제약조건 기반 물리에 Ball-socket Joint, Hinge Joint,
Cone-twist Joint의 관절 제약조건을 추가

hop 수가 2 이하인 Body와는 Self-collision 무시

단진자, 체인, 힌지, 휴머노이드, 캐릭터 순으로

관절 복잡도를 올리며 구현

⇒ 다수의 적이 동시에 죽어도 단조롭지 않도록 연출



래그돌화된 적 몬스터들



테스트용 단진자, 힌지, 휴머노이드 객체

03 PROJECT | 이방인(Outlander) – 주요 기여

Ragdoll Physics – Cone-twist Joint 사례

```
void ConeTwistJoint::solveVelocity()
{
    // Damping is NOT applied here -- it runs once per sub-step in prepare().
    // See kJointDampingRate for why per-iteration damping was wrong.

    // --- Translational ---
    {
        const mu::Vec3 rv      = relVelAt(bodyA_, bodyB_, cache_.rA, cache_.rB);
        const mu::Vec3 cVel     = rv + cache_.linPseudoBias;
        const mu::Vec3 dLambda = -(cVel * cache_.linEffMassInv);
        cache_.linAccImp = cache_.linAccImp + dLambda;
        applyLinearImpulsePair(bodyA_, bodyB_, dLambda, cache_.rA, cache_.rB);
    }

    // --- Cone ---
    if (cache_.coneActive) {
        JacobianRow row;
        row.bodyA = bodyA_;
        row.bodyB = bodyB_;
        row.angA = cache_.coneAxis;
        row.angB = -cache_.coneAxis;
        row.effMass = cache_.coneEffMass;
        row.rhs = cache_.coneBias;
        row.lower = 0.f;
        solveJacobianRow(row, cache_.coneAccImp, kAngSOR);
    }

    // --- Twist ---
    if (cache_.twistActive) {
        JacobianRow row;
        row.bodyA = bodyA_;
        row.bodyB = bodyB_;
        row.angA = cache_.twistAxis;
        row.angB = -cache_.twistAxis;
        row.effMass = cache_.twistEffMass;
        row.rhs = cache_.twistBias;
        row.lower = cache_.twistLo ? -std::numeric_limits<float>::infinity() : 0.f;
        row.upper = cache_.twistHi ? 0.f : std::numeric_limits<float>::infinity();
        solveJacobianRow(row, cache_.twistAccImp, kAngSOR);
    }
}
```

Velocity solver

```
void ConeTwistJoint::solvePosition()
{
    {
        const mu::Vec3 rv      = pseudoRelVelAt(bodyA_, bodyB_, cache_.rA, cache_.rB);
        const mu::Vec3 cVel     = rv + cache_.linPseudoBias;
        const mu::Vec3 dLambda = -(cVel * cache_.linEffMassInv);
        cache_.linPseudoAccImp = cache_.linPseudoAccImp + dLambda;
        applyPseudoLinearImpulsePair(bodyA_, bodyB_, dLambda, cache_.rA, cache_.rB);
    }

    // Cone angular position correction via split impulse (pseudo-omega).
    // This runs in addition to the velocity-level Baumgarte correction in
    // solveVelocity() and directly adjusts body orientations without injecting
    // energy into the real velocity state.
    if (cache_.coneActive) {
        JacobianRow row;
        row.bodyA = bodyA_;
        row.bodyB = bodyB_;
        row.angA = cache_.coneAxis;
        row.angB = -cache_.coneAxis;
        row.effMass = cache_.coneEffMass;
        row.rhs = cache_.conePseudoBias;
        row.lower = 0.f;
        row.pseudo = true;
        solveJacobianRow(row, cache_.conePseudoAccImp);
    }

    // Twist angular position correction.
    if (cache_.twistActive) {
        JacobianRow row;
        row.bodyA = bodyA_;
        row.bodyB = bodyB_;
        row.angA = cache_.twistAxis;
        row.angB = -cache_.twistAxis;
        row.effMass = cache_.twistEffMass;
        row.rhs = cache_.twistPseudoBias;
        row.lower = cache_.twistLo ? -std::numeric_limits<float>::infinity() : 0.f;
        row.upper = cache_.twistHi ? 0.f : std::numeric_limits<float>::infinity();
        row.pseudo = true;
        solveJacobianRow(row, cache_.twistPseudoAccImp);
    }
}
```

Position solver

```
float solveJacobianRow(const JacobianRow& row, float& accImpulse, float sor)
{
    auto linVel = [&](const RigidBody* b) {
        return row.pseudo ? b->pseudoLinearVel() : b->linearVel();
    };
    auto angVel = [&](const RigidBody* b) {
        return row.pseudo ? b->pseudoOmega() : b->omega();
    };

    const float Jv =
        mu::dot(row.linA, linVel(row.bodyA)) + mu::dot(row.angA, angVel(row.bodyA)) +
        mu::dot(row.linB, linVel(row.bodyB)) + mu::dot(row.angB, angVel(row.bodyB));

    const float rawDelta = -(Jv + row.rhs) * row.effMass;
    const float prev = accImpulse;
    const float candidate = std::clamp(prev + rawDelta, row.lower, row.upper);
    const float delta = (candidate - prev) * sor;
    accImpulse = prev + delta;
    if (delta == 0.f)
        return 0.f;

    if (row.bodyA->invMass() > 0.f) {
        const mu::Vec3 dv = row.linA * (delta * row.bodyA->invMass());
        const mu::Vec3 dw = (row.angA * delta) * row.bodyA->invInertiaWorld();
        if (row.pseudo) {
            row.bodyA->addPseudoLinearVel(dv);
            row.bodyA->addPseudoOmega(dw);
        } else {
            row.bodyA->setLinearVel(row.bodyA->linearVel() + dv);
            row.bodyA->setOmega(row.bodyA->omega() + dw);
        }
    }

    if (row.bodyB->invMass() > 0.f) {
        const mu::Vec3 dv = row.linB * (delta * row.bodyB->invMass());
        const mu::Vec3 dw = (row.angB * delta) * row.bodyB->invInertiaWorld();
        if (row.pseudo) {
            row.bodyB->addPseudoLinearVel(dv);
            row.bodyB->addPseudoOmega(dw);
        } else {
            row.bodyB->setLinearVel(row.bodyB->linearVel() + dv);
            row.bodyB->setOmega(row.bodyB->omega() + dw);
        }
    }

    return delta;
}
```

Jacobian helper

03 PROJECT | 이방인(Outlander) – 주요 기여

Image Based Lighting

환경 큐브맵을 광원으로 사용하는 간접광, Split-sum 근사 적용

$$\int L_i(l) \cdot f_r(l, v) \cdot (n \cdot l) dl \approx \left(\int L_i(l) \cdot D(l, v) \cdot (n \cdot l) dl \right) \cdot \left(\int f_r(l, v) \cdot (n \cdot l) dl \right)$$

로드 시 3종류의 데이터 미리 계산

- Irradiance Cube : Cosine Convolution, diffuse 항 (32^2)
- Prefiltered Env Cube : GGX Importance Sampling 256샘플,
mip level = roughness (128^2 , 5 mip)
- BRDF LUT : Split-sum 2차항 테이블, F0 스케일/바이어스 (256^2)

컬링 런타임 비용은 큐브 2회 + LUT 1회 샘플링으로 고정,
Deferred Lighting Pass에서 직접광에 가산

IBL Diffuse/IBL Specular/BRDF LUT를
개별 출력하는 디버그 뷰 구현

⇒ 단순 전역 ambient 상수항에서 발전하여
보다 사실적인 간접광 연출

In-game Scene



IBL Diffuse
IBL Specular



03 PROJECT | 이방인(Outlander) – 주요 기여

Image Based Lighting – HLSL

```
[numthreads(8, 8, 1)]
void CSMain(uint3 id : SV_DispatchThreadID) {
    if (id.x >= faceRes || id.y >= faceRes) return;
    uint face = id.z;

    float2 uv = (float2(id.xy) + 0.5f) / float(faceRes);
    float3 N = dirFromFaceUV(face, uv);

    // Build a tangent frame around N.
    float3 up = abs(N.y) < 0.999f ? float3(0.0f, 1.0f, 0.0f) : float3(1.0f, 0.0f, 0.0f);
    float3 T = normalize(cross(up, N));
    float3 B = cross(N, T);

    float3 irradiance = float3(0.0f, 0.0f, 0.0f);
    float nrSamples = 0.0f;
    const float sampleDelta = 0.025f;

    for (float phi = 0.0f; phi < 2.0f * PI; phi += sampleDelta) {
        for (float theta = 0.0f; theta < 0.5f * PI; theta += sampleDelta) {
            // tangent-space sample -> world
            float3 ts = float3(sin(theta) * cos(phi), sin(theta) * sin(phi), cos(theta));
            float3 sampleVec = ts.x * T + ts.y * B + ts.z * N;
            irradiance += sampleEnv(sampleVec) * cos(theta) * sin(theta);
            nrSamples += 1.0f;
        }
    }
    irradiance = PI * irradiance / max(nrSamples, 1.0f);

    dstCube[uint3(id.xy, face)] = float4(irradiance, 1.0f);
}
```

IBL Diffuse (Irradiance)

```
[numthreads(8, 8, 1)]
void CSMain(uint3 id : SV_DispatchThreadID) {
    if (id.x >= faceRes || id.y >= faceRes) return;
    uint face = id.z;

    float2 uv = (float2(id.xy) + 0.5f) / float(faceRes);
    float3 N = dirFromFaceUV(face, uv);
    float3 R = N;
    float3 V = N;

    const uint SAMPLE_COUNT = 256u;
    float3 prefiltered = float3(0.0f, 0.0f, 0.0f);
    float totalWeight = 0.0f;

    for (uint i = 0u; i < SAMPLE_COUNT; ++i) {
        float2 Xi = hammersley(i, SAMPLE_COUNT);
        float3 H = importanceSampleGGX(Xi, N, roughness);
        float3 L = normalize(2.0f * dot(V, H) * H - V);

        float NdotL = max(dot(N, L), 0.0f);
        if (NdotL > 0.0f) {
            prefiltered += sampleEnv(L) * NdotL;
            totalWeight += NdotL;
        }
    }
    prefiltered = prefiltered / max(totalWeight, 1e-4f);

    dstCube[uint3(id.xy, face)] = float4(prefiltered, 1.0f);
}
```

IBL Specular (Prefiltered)

```
float3 computeIBL(float3 N, float3 V, float3 albedo, float roughness, float metallic, float ao) {
    float NdotV = max(dot(N, V), 0.f);
    float3 R = reflect(-V, N);

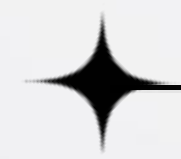
    float3 F0 = lerp(float3(0.04f, 0.04f, 0.04f), albedo, metallic);
    float3 kS = fresnelSchlickRoughness(NdotV, F0, roughness);
    float3 kD = (1.f - kS) * (1.f - metallic);

    // Diffuse irradiance.
    float3 irradiance = sampleBindlessCube(idxIrradiance, N).rgb;
    float3 diffuse = irradiance * albedo;

    // Specular prefiltered reflection + BRDF LUT (split-sum).
    float maxMip = float(max(prefilteredMipCount, 1u) - 1u);
    float3 prefiltered = sampleLevelBindlessCube(idxPrefiltered, R, roughness * maxMip).rgb;
    float2 brdf = sampleBindless(idxBRDFLUT, float2(NdotV, roughness)).rg;
    float3 specular = prefiltered * (F0 * brdf.x + brdf.y);

    // [Temp] 어차피 보스만 ao map이 있으므로, 보스가 자연스러운 렌더링이 되도록만 수치를 맞춘다.
    return (kD * diffuse + specular) * (1.f - min(ao * ao, 0.25f)) * iblIntensity;
}
```

IBL Lighting



03 PROJECT | 이방인(Outlander) – 주요 기여

HDR Bloom

선형 HDR 기반 발광 표현

모든 조명 결과를 SceneColor 버퍼 R16G16B16A16_FLOAT 에 누적
Exposure → ACES 톤매핑 → Gamma 보정

Soft-knee Threshold로 밝은 영역만 추출

→ 13-tap Downsample 밍 체인(최대 6단계)

→ 3x3 Tent Filter Additive Upsample로 역순 합성

통합된 발광 연출 구현 경로

⇒ 에너지 오브, 테두리 강조, 무기 Emmission,
경로 리본 메시, 보스 위압 연출에 활용



Bloom 예시 – 에너지 오브, 무기 Emmission, 경로 리본 메시

03 PROJECT | 이방인(Outlander) – 주요 기여

GPU Driven Rendering

GPU 기반 컬링 기법인 Hi-Z Occlusion Culling를 적용,
ExecuteIndirect(...), Command Buffer 등 Indirect
Drawing API를 활용해 GPU에서 Drawcall 생성

컬링 결과를 CPU로 readback해 애니메이션/물리 연산 생략을
통해 게임 업데이트 로직까지 최적화

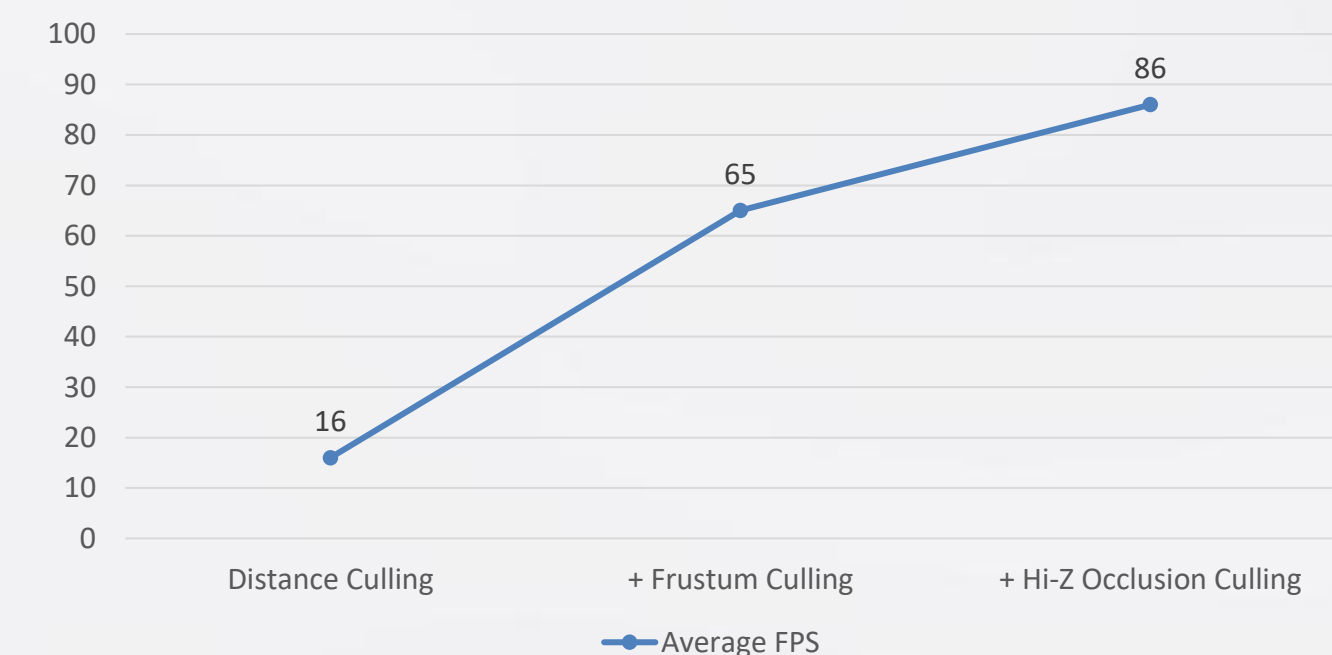
Distance Culling → Frustum Culling → Hi-Z Culling

⇒ 월드가 확장되고 객체가 늘어나도 화면 내 보이는
객체 수에 변화가 없다면 안정된 성능 유지



인게임 컬링 지표 (인스턴스 수)

컬링 성능 지표(FPS)



03 PROJECT | 이방인(Outlander) – 주요 기여

GPU Driven Rendering

```
// 4. Hi-Z Command Pass
DISPLAY_ERROR_DX_VOID(cmdList->SetPipelineState(hiZCommandShader_.Get()), false);
pResources_>hiZPass.perFrameDataCommand.bindCompute(cmdList, rootParamIdxPFD_, roomId_);
pResources_>hiZPass.perGroupData.bindComputeAsSRV(cmdList, rootParamIdxPerGroupData_, roomId_);
pResources_>hiZPass.indirectCmd.bindCompute(cmdList, rootParamIdxIndirectCmd_, roomId_);

// Command 패스는 그룹당 1엔트리 기록(스캔 아님)이라 multi-threadgroup이 정상.
static constexpr auto commandDispatchUnit = 64u;
DISPLAY_ERROR_DX_VOID( cmdList->Dispatch(
    static_cast<u32t>( (groupCnt + commandDispatchUnit - 1u) / commandDispatchUnit ), 1u, 1u
), false );

pResources_>hiZPass.indirectCmd.uavBarrier(cmdList, roomId_);

transitionResourceState(cmdList,
    pResources_>hiZPass.visibleIndices.resource(roomId_),
    D3D12_RESOURCE_STATE_UNORDERED_ACCESS,
    D3D12_RESOURCE_STATE_NON_PIXEL_SHADER_RESOURCE);
transitionResourceState(cmdList,
    pResources_>hiZPass.indirectCmd.resource(roomId_),
    D3D12_RESOURCE_STATE_UNORDERED_ACCESS,
    D3D12_RESOURCE_STATE_INDIRECT_ARGUMENT);

// visibility feedback readback: cull이 기록한 이번 프레임 슬롯을 단일 readback 리소스의
// 같은 슬롯 offset으로 복사한다. cull(u3) 쓰기 이후 어떤 패스도 visibilityFeedback을
// 건드리지 않으므로(여전히 UAV 상태), copy의 UAV->COPY_SOURCE 전환이 곧 쓰기 완료 동기화점이다.
// 전용 fence는 걸지 않는다 - false negative만큼은 일어나지 않는다.
{
    const u64t slotBytes = static_cast<u64t>(Resources::HiZPass::MAX_HIZ_INSTANCES) * sizeof(u32t);
    const u64t slotOffset = visibilitySlot_ * slotBytes;
    pResources_>hiZPass.visibilityFeedback.copyToReadback(
        cmdList, 0u, gBufferEvents_.size() * sizeof(u32t),
        D3D12_RESOURCE_STATE_UNORDERED_ACCESS, slotOffset, slotOffset
    );
    pResources_>hiZPass.slotEntryCount[visibilitySlot_] = static_cast<u32t>(gBufferEvents_.size());
}
```

Hi-Z Occlusion Culling Command Pass CPU-side (C++)

```
1 struct PerGroupData {
2     uint instCnt;
3     uint idxCnt;
4     uint groupOffset;
5     uint padding;
6 };
7
8 struct DrawIndexedInstancedArgs
9 {
10     uint IndexCountPerInstance;
11     uint InstanceCount;
12     uint StartIndexLocation;
13     int BaseVertexLocation;
14     uint StartInstanceLocation;
15 };
16
17 struct IndirectCommand
18 {
19     uint groupOffset;
20     DrawIndexedInstancedArgs drawArgs;
21 };
22
23 cbuffer PerFrameData : register(b1) {
24     uint groupCnt;
25     uint3 padding1;
26 };
27
28 StructuredBuffer<PerGroupData> gGroups : register(t5);
29 RWStructuredBuffer<IndirectCommand> gOutArgs : register(u0, space1);
30
31 [numthreads(64, 1, 1)]
32 void CSMain(uint3 id : SV_DispatchThreadID)
33 {
34     uint idx = id.x;
35     if (idx >= groupCnt)
36         return;
37
38     gOutArgs[idx].groupOffset = gGroups[idx].groupOffset;
39     gOutArgs[idx].drawArgs.IndexCountPerInstance = gGroups[idx].idxCnt;
40     gOutArgs[idx].drawArgs.InstanceCount = gGroups[idx].instCnt;
41     gOutArgs[idx].drawArgs.StartIndexLocation = 0u;
42     gOutArgs[idx].drawArgs.BaseVertexLocation = 0;
43     gOutArgs[idx].drawArgs.StartInstanceLocation = 0;
44 }
```

Hi-Z Occlusion Culling Command Pass GPU-side (HLSL)

03 PROJECT | 이방인(Outlander) – 주요 기여

Cascaded Shadow Mapping

카메라로부터의 거리에 따라 방향광의 투영 행렬을 다르게 설정하여 복수의 그림자 맵 생성, 그림자의 LOD를 구현

$$C_i = \lambda \cdot n \left(\frac{f}{n} \right)^{\frac{i}{N}} + (1 - \lambda) \cdot (n + (f - n) \cdot \frac{i}{N})$$

서로 다른 Cascade Level의 경계($\alpha=15\%$)에서 선형 보간을 통해 Popping 효과 방지

최종적으로 5x5 픽셀 PCF Percentage Closer Filtering 을 적용해 부드러운 그림자 연출

⇒ 광활한 크기의 맵에서 거리에 관계없이 부드러운 그림자 연출



인게임 그림자

Cascade Level 디버그 뷰



03 PROJECT | 이방인(Outlander) – 주요 기여

Skill System

Lua script 활용 스킬 정의

“PlayAnimation”, “PlaySound”, “PlayVFX”,
“SpawnHitbox”, “DestroyHitbox”의 이벤트 타입

이벤트의 발생 시간과 함께 연관 에셋, 수치들을 지정하는
타임 테이블

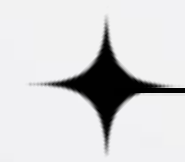
스킬을 시연하며 실시간으로 수치를 조정할 수 있는 별도 툴 제작

클라이언트와 서버 모두 참조,
⇒ 스킬 제작 효율성 증대 및
스킬 밸런스 조절의 빌드 의존성 제거



스킬 에디터

스킬 사양 lua 스크립트



03 PROJECT | 이방인(Outlander) – 주요 기여

Animation Blending

4방향 걷기, 달리기 Keyframe Animation nlerp 보간
피격, 사망의 경우 이벤트 발생 후 경과 시간에 따라
이동계열 애니메이션과 가장 마지막에 slerp 보간

Masking을 통해 특정 본 그룹에 대해 애니메이션 레이어링 가능
카메라 pitch에 따른 spine 본 조정 및 상하체 애니메이션 분리
플레이어와의 거리, 마지막 갱신으로부터 경과한 시간을
기준으로 애니메이션 갱신 우선순위 설정

일정 거리 이상의 객체에 대해선 baked animation 재생

마우스 에임에 맞는 모션,
⇒ 자연스러운 이동 중 공격 모션 구현
애니메이션 연산을 최악 10ms 안으로 최적화

```
// 스파인 체인(spine_01..03)에 조준 pitch를 분산 주입한다.  
// 각 체인 본 k의 피벗(드레스 공간 본 원점) 기준 X축 회전 K를 서브트리 전체에  
// 우측 곱(dress[b] * K)으로 적용한다 - 부모→자식 누적이 끝난 상태이므로 서브트리  
// 전 본에 곱해야 자식이 함께 회전한다. 3관절에 pitch/N씩 누적해 자연스러운 곱힘.  
// (aimPitch += 아래를 봄 = 상체 숙임. rotateXH(+))가 전방을 아래로 기울임 - Phase 2 검증됨.)  
void AnimBlenderPlayer::applyAimPitch() {  
    // 사망 페이드: death 블렌딩과 함께 조준 자세도 풀어준다.  
    const float effPitch = aimPitch_ * (1.f - tDeath_);  
    if (std::fabs(effPitch) < 1e-4f) return;  
  
    auto& dress = dressXformData();  
  
    int chainCnt = 0;  
    for (int c = 0; c < 3; ++c) {  
        if (spineChainIdx_[c] >= 0) ++chainCnt;  
    }  
    const auto perJoint = mu::Radian(effPitch / static_cast<float>(chainCnt));  
  
    for (int c = 0; c < chainCnt; ++c) {  
        const int k = spineChainIdx_[c];  
        // 앞 단계 K가 이미 반영된 dress[k]에서 피벗을 얻는다 (체인 누적 순서 중요).  
        const mu::Vec3 pivot = mu::Vec3(mu::Vec4(0.f, 0.f, 0.f, 1.f) * dress[k]);  
        const mu::Mat4x4 K = mu::translate(-pivot) * mu::rotateXH(perJoint) * mu::translate(pivot);  
        for (std::size_t b = 0; b < dress.size(); ++b) {  
            if (spineDepth_[b] > c) {  
                dress[b] = dress[b] * K;  
            }  
        }  
    }  
}  
  
// 애니메이션의 프레임들을 블렌딩한다.  
for (std::size_t i = 0; i < framesBlended_.size(); ++i) {  
    WeightedAnimFrame frames[] = {  
        WeightedAnimFrame{ .frame = framesIdle[i], .w = tIdle_ },  
        WeightedAnimFrame{ .frame = framesRunForward[i], .w = tRunForward_ },  
        WeightedAnimFrame{ .frame = framesRunBackward[i], .w = tRunBackward_ },  
        WeightedAnimFrame{ .frame = framesRunLeft[i], .w = tRunLeft_ },  
        WeightedAnimFrame{ .frame = framesRunRight[i], .w = tRunRight_ },  
    };  
    framesBlended_[i] = sumWeightedAnimFrames(frames);  
    // 공격 오버레이 보간 - 상하체 마스크 적용.  
    // 정지(tIdle_=1) 시 전 본 가중치가 tAttack_이 되어 종전 전신 공격과 동일하고,  
    // 이동 시 하체(mask=0)는 run 클립을 유지해 발 미끄러짐이 사라진다.  
    if (framesAttack) {  
        const float m = upperBodyWeight(i);  
        const float wAttack = tAttack_ * (m + (1.f - m) * tIdle_);  
        framesBlended_[i] = lerpAnimFrames(framesBlended_[i], (*framesAttack)[i], wAttack);  
    }  
    // hit animation 보간 (nlerp 쓰면 팔꿈치 꼬임)  
    framesBlended_[i] = lerpAnimFrames(framesBlended_[i], framesHit[i], tHit_);  
    // death animation 보간  
    framesBlended_[i] = lerpAnimFrames(framesBlended_[i], framesDeath[i], tDeath_);  
}  
std::ranges::transform(framesBlended_, localXforms.begin(), convertAnimFrameToMatrix);
```

카메라 pitch에 따른 spine 본 조정

키프레임 애니메이션 블렌딩

03 PROJECT | 이방인(Outlander) – 주요 기여

Animation Scheduling

플레이어와의 거리, 마지막 갱신으로부터 경과한 시간을
기준으로 애니메이션 갱신 우선순위 설정

일정 거리 이상의 객체에 대해선 baked animation 재생

⇒ 애니메이션 연산을 최악 10ms 안으로 최적화

```
void AnimBlender::updatePriority(PassKey<AnimSystem>, Seconds dt, mu::Vec3 refPos) {
    const float dist = (cachedPos_ - refPos).len();

    constexpr float kDistScale = 50.f;
    const float d = dist / kDistScale;

    // 거리 기반 weight
    const float wDist = 1.f / (1.f + d * d);

    // 거리 LOD: refPos에서 약 29m(= kDistScale * 0.577) 밖이면 baked 샘플로 전환한다.
    mode_ = wDist < 0.75f ? Mode::Baked : Mode::Keyframe;

    // 시간 기반 weight
    const float t = static_cast<float>(updateLag_.count());
    constexpr float kTimeScale = 0.5f; // 0.2~0.5

    // saturating 형태 (폭주 방지)
    const float wTime = 1.f + (t / (t + kTimeScale));

    // 최종 priority 누적
    priority_ = wDist * wTime;

    // lastUpdateTime_ 누적
    updateLag_ += dt;
}
```

애니메이션 우선순위 갱신

```
void AnimSystem::update(Seconds timeSlice) {
    // culled 블렌더를 뒤로 밀고, visible range([begin, visibleEnd))만 처리한다.
    const auto visibleEnd = std::partition(blenders_.begin(), blenders_.end(),
        [](AnimBlender* b) { return !b->isCulled(); });
    const std::size_t visibleCount =
        static_cast<std::size_t>(std::distance(blenders_.begin(), visibleEnd));

    if (visibleCount == 0u) return;

    auto tp = HighResolutionClock::now();
    Seconds elapsed = 0s;

    // priority 기준 max-heapify (visible range만)
    std::ranges::make_heap(
        std::ranges::subrange(blenders_.begin(), visibleEnd),
        std::less<>(), &AnimBlender::priority);

    std::size_t cntProcessed = 0u; // 처리한 블렌더 수
    std::size_t jobCnt = 0u; // 처리한 job 수
    while (elapsed < timeSlice && cntProcessed < visibleCount) {
        // 남은 블렌더의 수가 jobSize_보다 작을 수 있다.
        // 그럴 경우엔 남은 블렌더의 수만큼 블렌더들을 처리하도록 한다.
        const auto iteration = std::min(jobSize_, visibleCount - cntProcessed);

        for (std::size_t i = 0; i < iteration; ++i) {
            // 힙의 끝은 "이번 프레임에 지금까지 뽑아낸 총 개수"만큼 줄어들어 있어야 한다.
            // (batch 경계를 넘을 때 i가 0으로 리셋되므로 cntProcessed를 더해야 한다.)
            // 이 누적을 빼먹으면 두 번째 batch부터 이미 처리한 상위 priority 블렌더를
            // 다시 pop_heap 범위에 포함시켜, 동일 블렌더를 중복 처리하고 하위 블렌더는
            //영영 갱신하지 못한다(거리상 멀지 않은데도 툭툭 끊기는 원인).
            const auto pHeapEnd = std::prev(visibleEnd, cntProcessed + i);
            std::pop_heap(blenders_.begin(), pHeapEnd);
            auto& blender = *std::prev(pHeapEnd);

            blender->onCalcLocal({});
            blender->setStage({}, AnimBlender::Stage::committedLocal);

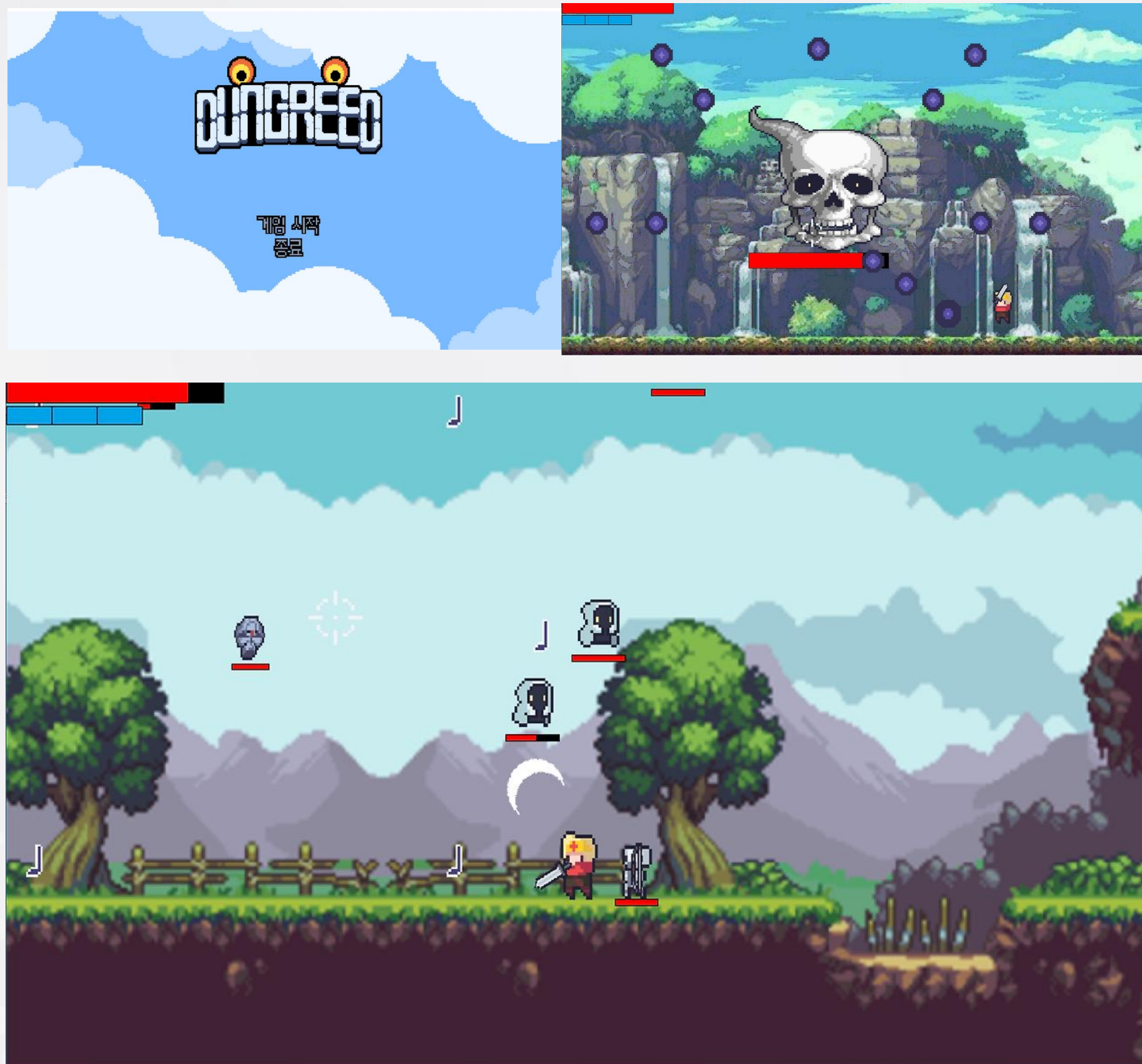
            blender->onCalcDress({});
            blender->setStage({}, AnimBlender::Stage::committedDress);

            blender->onCalcFinal({});
            blender->setStage({}, AnimBlender::Stage::committedFinal);
        }

        cntProcessed += iteration;
        ++jobCnt;
        elapsed = HighResolutionClock::now() - tp;
    }
}
```

애니메이션 시스템 업데이트

04 PROJECT | Dungreed 모작



Dungreed 모작

'윈도우 프로그래밍' 과목 팀 프로젝트

장르 : 2D 횡스크롤 액션 게임

사용 언어 : C++14

개발 환경 : Visual Studio 2019, Win32 API, Git

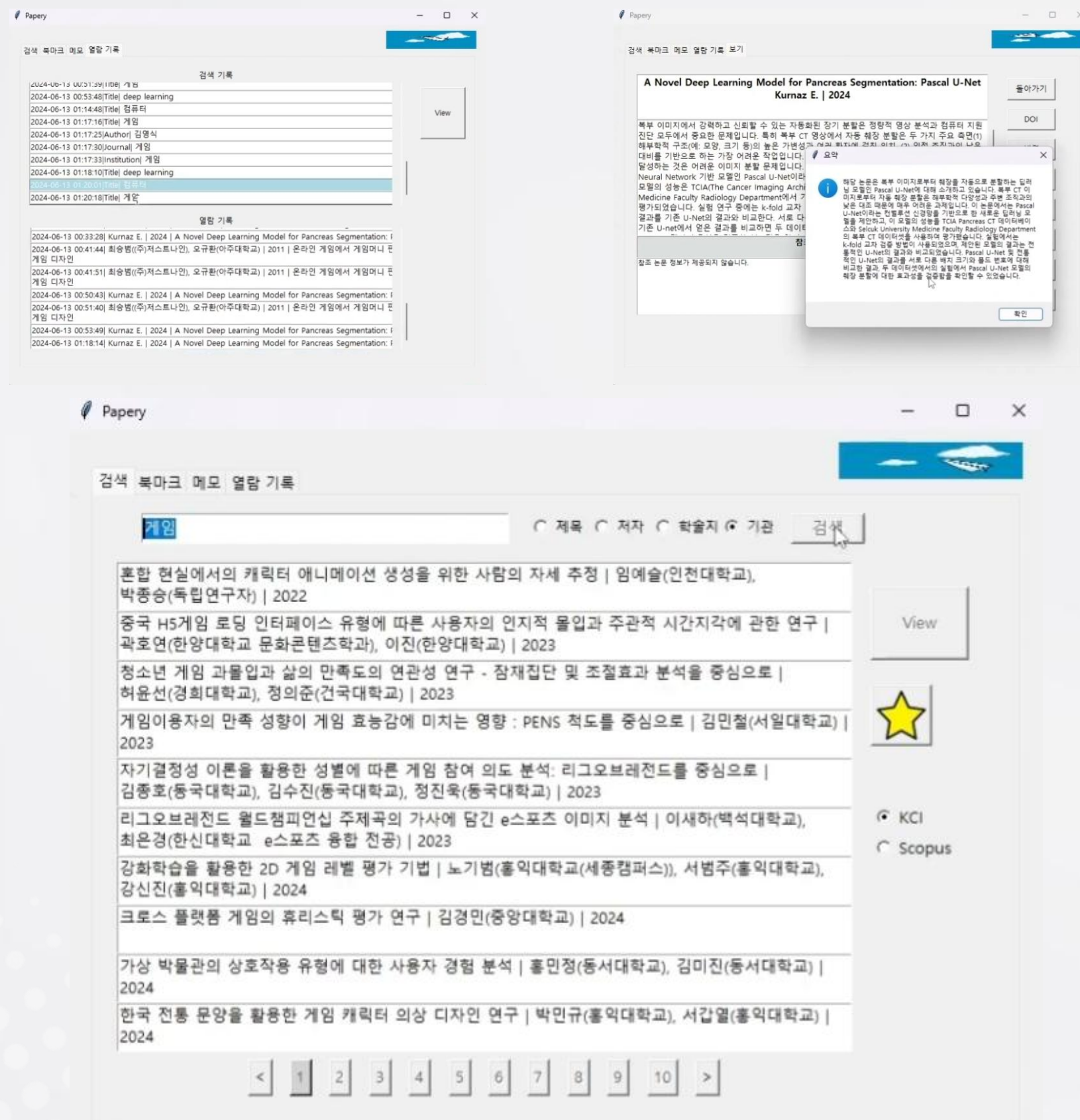
개발 기간 : 2021.05.-2021.06.

개발 인원 : 2인

담당 역할 : 애니메이션 구현, 발사체 구현, 충돌 처리 구현, 몬스터 ai 구현, 사운드 구현, 플레이어 이동 모션 개선, 레벨 데이터 정의 및 추출



05 PROJECT | Papery



Papery  

‘스크립트 언어’ 과목 팀 프로젝트

종류 : 논문 통합 검색 GUI 프로그램

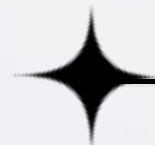
사용 언어 : python 3.12

개발 환경 : VS Code, Tkinter

개발 기간 : 2024.05.-2024.06.

개발 인원 : 2인

담당 역할 : 각 API 활용 검색 데이터 수집 및 페이지 정렬 구현, 논문 AI 요약 구현, 검색 로그 구현, 해외 논문 번역 기능 구현, 로딩창 구현, G-mail 및 Telegram 연동 구현



Thank you

읽어주셔서 감사합니다.

CONTACT

+82 010 6477 4881

wkdauddns9@gmail.com